# A constructive, executable decision procedure for linear temporal logic in Agda

Proposal for a master's thesis

October 26, 2026

**Starting point.** The student has followed an elementary course in Agda; no prior knowledge of linear temporal logic (LTL) is assumed. The project starts by learning LTL and its usual decidability proof, then develops an existing Agda formalisation into a clearer, executable, proof-producing procedure for LTL including *Until*. A further objective is to understand the constructive status of the semantic statements used in the classical proof.

**Existing development.** The Agda sources and accompanying mathematical notes are available at

`https://github.com/coquand/LTL`.

The development treats Boolean connectives, *next* ($\mathsf{X}$), and *always* ($\mathsf{G}$); *eventually* is an abbreviation $\mathsf{F}A = \neg\mathsf{G}\neg A$. Until is not yet included. There are two decision procedures, based respectively on state elimination and strongly connected components (SCCs), with a comparison suite of 47 examples. Their result has the form

$$\mathrm{Deriv}(A) \;+\; \mathrm{LassoCountermodel}(A).$$

Thus a positive answer carries a derivation in a temporal proof system, and a negative answer carries a finite countermodel represented by a prefix and a repeating cycle, together with a refutation of derivability. The files `m4.pdf` and `m5.pdf` explain the two constructions. The sources use Agda's `-safe -without-K -exact-split` options and do not depend on the standard library.

## 1 First stage: learn LTL and the usual decidability proof

The first task is mathematical study, before changing the formalisation. The primary reference is Ben-Ari's *Mathematical Logic for Computer Science* [1], especially the chapters entitled *Temporal Logic: Formulas, Models, Tableaux* and *Temporal Logic: A Deductive System*. These are Chapters 13 and 14 in the third edition. The existing development refers to Chapters 11 and 12 of an earlier edition; chapter titles should be used to match the material across editions.

The student should learn:

1. The syntax and semantics of LTL on infinite words: Boolean connectives, next, always, eventually, and Until; validity, satisfiability, and evaluation on a specified word.
2. Ben-Ari's deductive system, representative derivations, and the soundness and completeness arguments, including extraction of a derivation from a failed tableau search.
3. The usual decidability argument: a finite closure of subformulas, locally consistent states, transitions, eventuality obligations, accepting components, and ultimately periodic models.

4. The automata perspective: translation to generalized Büchi automata and reduction to an emptiness problem. Vardi's survey and the construction of Gerth et al. provide complementary accounts [2, 3].

An initial deliverable is a short written account of the standard proof, with several tableaux and lasso models worked out by hand. The student should be able to explain why a cycle satisfying all local transition conditions can nevertheless fail an eventuality obligation. This prepares the distinction between an arbitrary loop and a fair loop that is central to the Agda development.

## 2 Understand and clarify the constructive proof

The next task is to read the mathematical notes and trace the main Agda interfaces. Useful entry points are `LTL.Formula`, `LTL.Deriv`, `LTL.Decide`, `LTL.DecideSCC`, `LTL.ShortPC`, and `LTL.Compare`. The desired account should explain how a finite search produces either a derivation or a countermodel, and where temporal induction enters the proof of refutability of eliminated states.

One concrete opportunity for simplification is the interface between a fair lasso and its correctness proof. Currently the SCC procedure still uses the original elimination computation on its countermodel branch to reuse a correctness theorem stated for the surviving states. A more general truth lemma for fair lassos of locally consistent states could remove that dependency. This is a candidate redesign to investigate; the project should choose the refactoring that best supports the Until extension rather than reorganise every module.

The mathematical explanations, module boundaries, and theorem statements are part of the contribution. A successful reorganisation should make the conditions on a countermodel explicit and reduce the amount of duplicated reasoning. Existing verified work on LTL-to-automata translation offers useful related work [4].

## 3 Extend the proof-producing procedure with Until

For an infinite valuation sequence $\sigma$, the intended positive clause is

$$\sigma, i \models A \,\mathsf{U}\, B \quad \Longleftrightarrow \quad \exists j \geq i \; (\sigma, j \models B \;\wedge\; \forall k \, (i \leq k < j \Rightarrow \sigma, k \models A)).$$

The student should extend the syntax, proof system, state construction, and correctness arguments. The unfolding law

$$A \,\mathsf{U}\, B \;\leftrightarrow\; B \vee (A \wedge \mathsf{X}(A \,\mathsf{U}\, B))$$

must be accompanied by an appropriate least-fixed-point principle in the proof system and a fairness condition in the search. Unfolding alone allows an obligation to be postponed forever. The relevant fairness predicate on a state $s$ is

$$(A \,\mathsf{U}\, B) \notin s \quad \vee \quad B \in s.$$

The truth lemma must also account for the fact that $A$ holds along the path before the occurrence of $B$.

The target is a total Agda procedure returning a derivation or a certified lasso countermodel for the extended language, with the hypotheses and metatheory stated explicitly. The existing proof system and derived connectives should be related carefully to any additional semantic connectives used when explaining Until.

## 4 Why the semantic interpretation matters constructively

The thesis should distinguish decidability of derivability, validity on explicitly presented lassos, satisfaction on a given arbitrary infinite word, and validity over all infinite words. It should identify the additional principles required by the classical semantic justification, with precise statements instead of a blanket identification of these decision problems.

A small formal example provides a useful starting point. Interpret eventually by constructive existence and disjunction by constructive disjunction. Validity over all Boolean streams of

$$\mathsf{F}p \vee \mathsf{G}\neg p$$

is equivalent to the limited principle of omniscience (LPO):

$$\forall \alpha : \mathbb{N} \to \{0,1\}, \qquad (\exists n\ \alpha(n) = 1)\ \vee\ (\forall n\ \alpha(n) = 0).$$

On a finite prefix and cycle the same alternative can be decided by finite inspection. Over arbitrary words, the witness-bearing clause for $\mathsf{F}p$ cannot simply be identified constructively with $\neg\mathsf{G}\neg p$. The student can formalise the equivalence with LPO and explain its metamathematical significance using, for example, Escardó's presentation [5]. Proving such an equivalence is different from proving independence of LPO within the formalisation.

This study should clarify why finite certificates suffice for the constructive syntactic result. For later comparison with S1S, Lichter and Smolka analyse the assumptions needed to pass from ultimately periodic to arbitrary-sequence semantics [6]. Their precise results concern S1S and Büchi automata; they should not be transferred to LTL without a separate argument.

## 5 Small examples and readable proofs

The implementation should be exercised throughout the project. An initial suite can include the existing examples and the following targets for the Until extension:

$$\begin{aligned}
&(p\ \mathsf{U}\ q) \to \mathsf{F}q,\\
&\mathsf{G}(p \to r) \to ((p\ \mathsf{U}\ q) \to (r\ \mathsf{U}\ q)),\\
&(p\ \mathsf{U}\ q) \leftrightarrow (q \vee (p \wedge \mathsf{X}(p\ \mathsf{U}\ q))).
\end{aligned}$$

These should yield derivations. The invalid converse $\mathsf{F}q \to (p\mathsf{U}q)$ should yield a countermodel. Further examples should include nested temporal operators, several eventuality obligations, and formula families whose size can be varied.

Decision time, construction of a certificate, and checking or printing it must be measured separately. The current development already illustrates that a small Boolean computation can conceal a large proof term. The comparison should record the Agda version, evaluation method, formula size, explored states, certificate size, and execution times; historical timings in the repository are starting information, not new measurements.

Readable proof output is an objective to evaluate on the examples. Suitable output would use named temporal lemmas, explicit invariants, and clearly identified calls to a proved propositional checker. Every displayed inference should reconstruct a checked derivation. Countermodels should display the prefix and cycle as valuations of the propositional variables. Concise output on all formulas is not assumed.

## 6 Scope and expected outcome

The core project comprises learning LTL and its standard proof, explaining the constructive argument, extending the Agda development with Until, and demonstrating the result on small

nontrivial examples. Refactoring should serve these goals. The final thesis should state exactly which results have been formalised and which improvements remain proposals.

The expected deliverables are a mathematical exposition, a documented Agda development with reproducible checking instructions, an executable example suite, and an evaluation of proof output and computational costs. The student should be able to explain the central invariants and justify the chosen representations and interfaces.

If progress permits, one further verified optimisation or a more systematic proof presentation can be added. Applying the approach to the existing S1S development is an ambitious extension; S2S is a longer-term research direction. Neither is required for completion of this thesis.

## References


[1] M. Ben-Ari, *Mathematical Logic for Computer Science*, 3rd edition, Springer, 2012. Temporal logic chapters, pp. 231–272. Publisher and electronic edition.

[2] M. Y. Vardi, *An automata-theoretic approach to linear temporal logic*, in F. Moller and G. Birtwistle (eds.), *Logics for Concurrency: Structure versus Automata*, LNCS 1043, Springer, 1996, pp. 238–266. Author's PDF.

[3] R. Gerth, D. Peled, M. Y. Vardi, and P. Wolper, *Simple on-the-fly automatic verification of linear temporal logic*, in *Protocol Specification, Testing and Verification XV*, 1995, pp. 3–18. Author's PDF.

[4] A. Schimpf, S. Merz, and J.-G. Smaus, *Construction of Büchi automata for LTL model checking verified in Isabelle/HOL*, TPHOLs 2009, LNCS 5674, pp. 424–439. Paper and bibliographic information.

[5] M. H. Escardó, *Omniscience*, Agda lecture material, 2011. Online Agda presentation of LPO.

[6] M. Lichter and G. Smolka, *Constructive Analysis of S1S and Büchi Automata*, 2018 preprint, arXiv:1804.04967. Paper and Coq development.

[7] *Constructive LTL decision procedures in Agda*, existing development and mathematical notes. `https://github.com/coquand/LTL`.